

PULS-EVENTS

Assistant intelligent de recommandation d'événements culturels

Rapport technique — Proof of Concept (RAG)

Architecture RAG · LangChain · Faiss · Mistral AI · FastAPI · Docker

Objectifs du projet

Mission Data Scientist pour Puls-Events — plateforme de recommandations culturelles



Contexte

Livrer un POC fonctionnel d'assistant conversationnel basé sur une architecture RAG, exposé via une API REST conteneurisée.



Problématique métier

Les LLM généralistes hallucinent et n'ont pas accès aux données événementielles temps réel ou localisées.



Objectif du POC

Démontrer la faisabilité du pipeline complet et valider la performance qualitative via une évaluation automatisée.



Périmètre

Bordeaux Métropole (Nouvelle-Aquitaine) — historique < 1 an et événements à venir, source Open Agenda.

Stack technique

Langage, orchestration, moteur vectoriel, LLM, API et infrastructure



Python 3.11

Langage & uv (lockfile)



LangChain (LCEL)

Orchestration RAG



Faiss (faiss-cpu)

IndexFlatL2 vectoriel



Mistral AI

mistral-embed / mistral-small



FastAPI + Uvicorn

API REST asynchrone



Pydantic v2

Validation de schéma



Pytest

Tests & évaluation



Docker Compose

Conteneurisation

Préparation et vectorisation des données



Source & nettoyage

- API OpenDataSoft / Open Agenda — filtre `location_city='Bordeaux'`, tri par `lastdate_begin`
- Élimination des enregistrements au titre ou à la description vides
- Normalisation des champs texte (titre, description, lieu, dates, adresse, ville)
- Stockage compressé au format Parquet (`events_raw.parquet`) via PyArrow
- Chaque ligne encapsule déjà une entité logique autonome et sémantiquement complète



Stratégie de chunking

`RecursiveCharacterTextSplitter` — chunks de 500 caractères, overlap de 50

1 ligne = 1 événement = 1 contexte complet. Le détail de la logique est présenté page suivante.



Embedding

- Modèle : `mistral-embed` — 1024 dimensions
- Vectorisation asynchrone par batchs (`MistralAIEmbeddings`)
- Détail du choix et positionnement page suivante

Focus — Stratégie de découpage (chunking)

RecursiveCharacterTextSplitter — chunk_size=500, chunk_overlap=50



Une coupe intelligente en cascade

L'algorithme tente d'abord de couper aux paragraphes ($\backslash n \backslash n$), puis aux phrases ($\backslash n$), puis aux mots — jamais au milieu arbitrairement.

Pourquoi pas de chunking sémantique / hiérarchique / agentique ?

- Les données sont déjà segmentées naturellement : 1 ligne = 1 événement = 1 contexte complet
- Les méthodes avancées (embeddings phrase par phrase pour détecter les ruptures de sens) seraient surdimensionnées et lentes pour de courtes fiches factuelles

Exemple concret — « Escape Hunt Bordeaux » (346 caractères)

346 < 500 → un seul chunk de 346 caractères, jamais fusionné avec l'événement suivant.



chunk_size = 500

Un plafond maximum, pas une taille imposée

- Densité sémantique ciblée : ~80-100 mots, la fenêtre idéale pour une unité d'info complète
- Un événement de 350 car. reste un chunk unique de 350 car. — jamais fusionné entre eux
- Trop grand → vecteur dilué ; trop petit → contexte insuffisant pour le LLM



chunk_overlap = 50

Un filet de sécurité contextuel

- Réservé aux rares descriptions longues (> 500 caractères)
- Répète la fin du segment précédent au début du suivant
- Empêche de scinder une entité critique (artiste, adresse, date) entre deux chunks

Focus — Choix du modèle de vectorisation

Positionnement dans les familles de vectorisation et justification du choix mistral-embed + Faiss



Approches lexicales / creuses

TF-IDF, BM25 — efficaces sur les mots exacts, mais aveugles aux synonymes.



Approches denses & contextuelles ★

Famille retenue — projection du texte dans un espace continu captant le sens profond.

Pourquoi mistral-embed (Mistral AI) ?



Excellence linguistique

Fort corpus francophone natif — essentiel pour les subtilités régionales et culturelles locales.



Dimensionnalité équilibrée

1024 dimensions : compromis parfait entre finesse sémantique et vitesse de calcul vectoriel.



Homogénéité de la stack

Même écosystème que mistral-small-latest — gouvernance des clés d'API et compatibilité LangChain simplifiées.

Moteur vectoriel Faiss & recherche exhaustive

faiss-cpu avec l'algorithme IndexFlatL2 — zéro infrastructure, précision absolue



Zéro infrastructure

Pas de base vectorielle cloud (Pinecone, Qdrant, Milvus) : Faiss est une bibliothèque C++ optimisée exécutée localement en RAM.



Recherche exacte

IndexFlatL2 = recherche exhaustive par distance euclidienne, équivalente à la similarité cosinus après normalisation.



Précision vs vitesse

Sur des millions de documents, on utiliserait l'ANN/HNSW pour la vitesse. Ici, l'exhaustivité tient en < 1 ms sans compromis.



Persistence légère

index.faiss (vecteurs) + index.pkl (métadonnées : titre, dates, adresse) — quelques mégaoctets, portable en conteneur.

Choix du modèle NLP

mistral-small-latest via l'API Mistral AI



Qualité

Excellent niveau de compréhension du français et respect rigoureux des instructions complexes.



Coût & latence

Modèle léger et économique, idéal pour un assistant conversationnel à faible latence.



Compatibilité

Intégration native dans l'écosystème LangChain via langchain-mistralai.



Template de prompt (anti-hallucination)

Température : 0.2

Tu es un assistant expert pour la plateforme Puls-Events, spécialisé dans la recommandation d'événements culturels. Réponds à la question de l'utilisateur de manière précise, chaleureuse et structurée, en te basant EXCLUSIVEMENT sur le contexte fourni ci-dessous. Si les informations fournies ne permettent pas de répondre, indique clairement l'absence d'événement correspondant dans le catalogue actuel.

API et endpoints exposés

FastAPI avec gestionnaire de cycle de vie asynchrone (lifespan)

Méthode	Route	Description	Succès / Erreur
GET	/health	Vérification de l'état opérationnel du service	200 OK / 500
POST	/ask	Ingestion d'une question, passage dans le RAG et retour de la recommandation	200 OK / 400 / 503
POST	/rebuild	Déclenchement de la ré-indexation complète de la base vectorielle à chaud	200 OK / 500

Exemple d'appel : `curl -X POST "http://localhost:8000/ask" -d '{"question": "Quels concerts de musique classique ?"}'`

Tests fonctionnels (tests/api_test.py) : 3 tests exécutés avec Pytest — 3 passed

Évaluation du système

Similarité sémantique cosinus entre réponse générée et vérité terrain (mistral-embed)

88,51 %

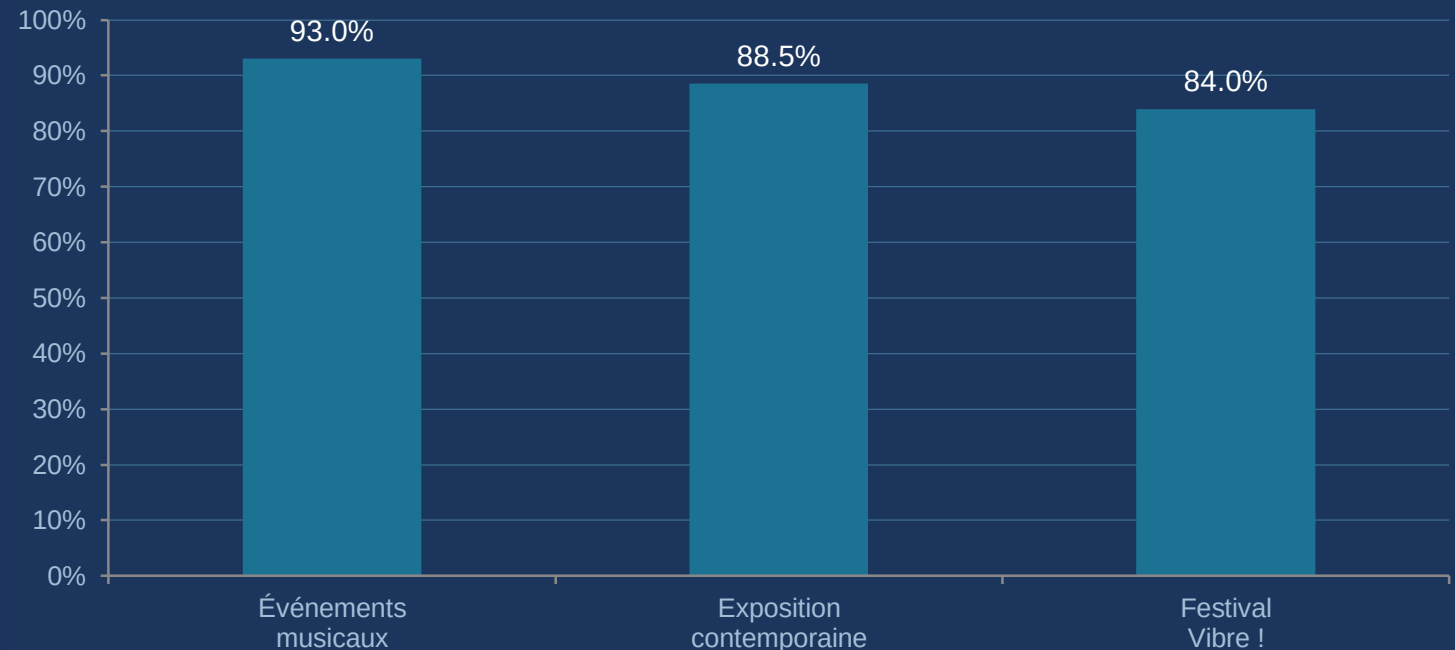
Similarité sémantique moyenne

Seuil de conformité : $\geq 0,75$



Zéro hallucination sur les requêtes hors catalogue

Résultats par cas de test



Cas positif — Concerts : le système extrait fidèlement le Quatuor Prométhée et le Festival Vibre !

Cas hors catalogue — Exposition : le modèle refuse explicitement de générer une fausse information.

Temps de réponse moyen observé : < 1,5 seconde

Recommandations et perspectives



Points forts

- Temps de réponse < 1,5 s en moyenne
- Zéro hallucination sur requêtes hors contexte
- Déploiement reproductible via Docker Compose



Limites actuelles

- Indexation limitée à l'échantillon Bordeaux
- Système stateless (pas d'historique multi-tours)
- Recherche dense sensible aux fautes de frappe



Axes d'amélioration V2

- Recherche hybride Faiss + BM25 (EnsembleRetriever)
- Re-ranking via Mistral Rerank
- Mémoire de session (cache Redis)
- Monitoring continu Ragas + CI/CD



Merci de votre attention

Puls-Events — Assistant intelligent de recommandation culturelle · POC RAG

Questions ?